Accelerating K-Means Clustering on GPUs

Problem Statement & Motivation

K-Means clustering is a fundamental technique in unsupervised machine learning that has a wide range of applications across many domains. Examples include customer segmentation in retail contexts, recommendation engines for platforms such as YouTube and Spotify, as well as fraud detection in banking.

The core computational bottleneck lies in the iterative assignment step, where the Euclidean distance between every point and every centroid is evaluated and assigned. As datasets expand into millions of points and beyond, computation on a single-threaded CPU becomes extremely slow. But because distance calculations for each of these data points are done independently from one another, this assignment exhibits strong data parallelism that is suitable for GPUs. The goal is to optimize for a reduction in runtime under the same workload, or strong scalability.

Proposed Methodology

First I will implement and measure the running time on a serial baseline. Then I will implement a CUDA kernel assigning one thread per data point, and stage centroid locations in GPU shared memory for broadcast reads. The centroid recomputation will be calculated directly on the GPU. I will verify the final outputs by asserting that the assignment labels for each point and the centroid coordinates match the serial baseline.

Hardware and Experimental Setup

I will use either the NCSU ARC compute nodes that are equipped with GPUs, or I will run it locally on the NVidia 1080 Ti that I have on my desktop.

As for the dataset, I'm still considering between several, but among them include the Individual Household Electric Power Consumption from the UCI Machine Learning Repository, which is used frequently as a standard benchmark for clustering.

Milestones

1. Environment and Baseline
2. Naiva CUDA Kernel
3. GPU optimizations
4. Scaling experiments and measurements
5. In-class presentation
6. Final report and code